

Question	Answer
is the initial embedding updated after every epoch? the vocabulary matrix is updated for lookup?	the vocabulary embedding matrix is trainable in normal LLM training. But it is generally updated after every optimizer step, not just after every epoch. The token IDs remain fixed; what changes is the embedding vector associated with those IDs. Those embeddings then become the input to the self-attention layer, which uses them to calculate Q, K, and V.
Wv is [2,0] so why its [1,1], while fetching the value of k?	it will be W_k, not W_v...
If every token attends to every other token, how does the model avoid being overwhelmed by irrelevant information?	Self-attention technically allows every token to look at every other token, but it does not mean every token gives equal importance to every other token.
does this mean each word in vocab will have multiple Weights for q,k,v?	Token ? one embedding vector ? projected through shared (per-head) Q/K/V matrices ? Q/K/V representations for that particular layer and position.
In a complex sentence, where we have two animals in a sentence, like dog and cat, how head 2 will behave ? will it still be itself?	Attention heads are fixed learned functions, not fixed rules. The head remains the same, but its attention weights dynamically change for every sentence and even for every token position.
Are these heads assigned the role of say looking at self or other words by humans or do the heads take on that role by themselves? If the latter, then what is stopping the 2 heads in this example from assuming the same role?	Attention heads are not assigned roles by humans. They learn different transformations through gradient descent. Specialization can emerge because different parameter sets discover useful patterns, but there is no hard rule preventing two heads from learning the same pattern.
How do we define what the head focuses on ?	We don't explicitly define what a head focuses on. Think of it as an emergent behavior.
How do we define what the head focuses on ?	The learned Q/K projection matrices determine what patterns cause tokens to attend to one another.
How do we define what the head focuses on ?	A sequence is the ordered stream of tokens that the Transformer processes together.
How do we define what the head focuses on ?	"The dog chased the cat."
How do we define what the head focuses on ?	After tokenization, suppose we get:
How do we define what the head focuses on ?	The dog chased the cat
What is sequence here? I didn't understand	we call it a sequence because order matters

In Quiz 1, if GPU and parallelization is removed from requirement, RNN will be better?	not necessarily, a transformer will be better thn RNN when it comes to learning better representation of the text. The GPU constraint is there as transformers take a lot of time and data to train
all token are assessed in parallel and then does the multi head attention done in parallel as well ?	Yes during training, the tokens in a sequence are processed in parallel, and the multiple attention heads are also computed in parallel. This is one of the major reasons Transformers are much faster to train than RNNs.
how to get the different weights here like W_q, W_k, W_v	They are simply learnable parameters of the Transformer, just like the embedding matrix, W_Q, W_K, W_V . are learned weight matrices. They are randomly initialized and then adjusted through backpropagation during training. We don't calculate them from the words; we learn them from the training data.
follow up to my previous Q- If the heads take on roles by themselves then how do researchers/ humans ascertain what aspect/ relationship each head has taken up after an LLM model is trained? If not so then will we be able to control how and what an LLM learns?	Yes, researchers can investigate what attention heads appear to be doing, but they generally don't directly assign a semantic role to a head.
when we say decoded till quick, what does it actually mean by term "decoded"?	The model has generated tokens up to the word/token quick, and those generated token IDs have been converted back into readable text.
During training we use encoders or decoders?	It depends on the model type: encoder-decoder models (like the original Transformer or T5) train both parts together, while GPT-style LLMs use only a decoder (trained to predict the next token) and BERT-style models use only an encoder (trained via masked-token prediction).
Why in decoder, attention is Q^*T . K but in Encoder $Q.K^*T$	That premise doesn't hold, both encoder and decoder attention use the same score formula, $\text{scores} = Q \cdot K^T$ (giving a $\text{seq_len} \times \text{seq_len matrix}$), so there's no standard Transformer variant where the decoder flips it to $Q^T \cdot K$.
during training also we dont reveal next work (teacher training). can you pls confirm?	Yes for a decoder-only LLM like Llama/GPT, during training we do NOT allow the model to see the future/target token when predicting it. This is exactly what the causal attention mask ensures.
i missed the point, please can you repeat why is masked a constraint in decoding but not in encoding. What is stopping us from seeing future tokens in decoding but not in encoding?	the key is what task the model is being trained to do. Nothing inherently prevents a decoder from seeing future tokens. We impose the causal mask because allowing future tokens would violate the next-token-prediction task. The encoder has no such constraint because its job is to understand the complete input, not predict it sequentially.

	Because in the decoder, self-attention is masked. If the current token is “cat,” it is only allowed to attend to itself and the tokens that came before it, not to future words that have not been generated yet. For example, if the output is “The cat sat on the mat”, when processing “cat”, the decoder can attend to “The” and “cat”, but not to “sat,” “on,” “the,” or “mat.” This prevents the model from “seeing the future” during training and keeps generation autoregressive. Translation is just the classic example, but cross-attention is also heavily used for vision-language, speech-language, multimodal models, and encoder-decoder architectures. yes, conceptually, with an important distinction between training and inference. For example, if the target generated so far is: “The cat” then X_{dec} contains the embeddings of “The” and “cat”: $X_{dec} = [\text{embedding}(\text{The}); \text{embedding}(\text{cat})]$ From this matrix, the decoder computes Q, K, and V for all available target tokens. During training, the whole target sequence can be put into the matrix at once, but the causal mask prevents each position from looking at future tokens. During actual generation, the matrix grows as each new token is generated. Not exactly Q, K, V themselves—those are computed each forward pass by multiplying the input embeddings by learned weight matrices (W_Q, W_K, W_V), so it's those weight matrices the model is actually learning; and yes, backpropagation computes gradients of the loss with respect to W_Q, W_K, W_V (and all other weights) and updates them via gradient descent to arrive at values that produce useful attention patterns.
why cat is not attending all word? Is cross attention also usually used for cases where no language translation is used? say in a case when a user asks Chat GPT a simple question in english? x decoder is matrix calculation, is matrix of embedding until the last word translated in target ?	
could you pls elaborate on this distinction pls?	
we learned so far that an LLM or any neural net based model is aiming to learn the weights/ relationships. In transformers are QKV the weights that the model is trying to learn during training? And is backpropagation working behind the scenes during training to arrive at optimum Q,K,V values?	

	During training, the decoder can still process all target tokens in parallel because the full target sequence is already available. A causal mask blocks each position from attending to future tokens. So “cat” can only see tokens up to “cat,” even though the matrix operations for all positions happen together on the GPU. During inference/generation, however, decoding is sequential: the model generates one new token, appends it, and then generates the next. So parallelism is high during training, but limited across token-generation steps during inference.
How does parallelism work in the decoders, as it can't look at future tokens?	The 4d usually comes from the hidden size of the Transformer's feed-forward network being set larger than the model embedding size. If the token representation has dimension $d_{\text{model}} = d$, then the FFN typically does: $d \rightarrow 4d \rightarrow d$. For example, if $d = 512$, the FFN may expand it to 2048 dimensions, apply a non-linearity, and then project it back to 512. The idea is that the larger intermediate space gives the model more capacity to learn complex transformations. 4d is mainly a design choice from the original Transformer, not a strict rule; modern models may use different expansion ratios.
where is the 4 coming from in RELU	Usually, the same type of normalization is used throughout a model, but each layer has its own normalization parameters.
is normalization fixed across all layers? is that a hyperparameter we set before training ?	The normalization operation/type is part of the architecture and is chosen before training. But its learnable parameters, such as LayerNorm's ? and ?, are learned during training. Usually each Transformer layer has its own normalization parameters rather than sharing one across all layers.
is normalization fixed across all layers? is that a hyperparameter we set before training ?	The choice of normalization method (LayerNorm, RMSNorm, etc.) and where it sits in each block (pre-norm vs post-norm) is a fixed architectural/hyperparameter decision made before training and is typically kept identical across all layers. However, each LayerNorm/RMSNorm instance still has its own learnable scale (and shift, if using LayerNorm) parameters per layer, so while the mechanism is uniform, the actual learned parameters differ layer to layer and are updated via backpropagation just like Q/K/V weights.

	The 4d usually comes from the hidden size of the Transformer's feed-forward network being set larger than the model embedding size. If the token representation has dimension $d_{\text{model}} = d$, then the FFN typically does: $d \rightarrow 4d \rightarrow d$. For example, if $d = 512$, the FFN may expand it to 2048 dimensions, apply a non-linearity, and then project it back to 512. The idea is that the larger intermediate space gives the model more capacity to learn complex transformations. 4d is mainly a design choice from the original Transformer, not a strict rule; modern models may use different expansion ratios.
What is 4d hidden units?	The dimension expansion itself is done by a learned linear layer (a weight matrix that maps d_{model}, e.g. 512, up to a larger d_{ff}, e.g. 2048). ReLU isn't what expands the dimension, it's just applied elementwise afterward to zero out negative values and introduce nonlinearity. Without that ReLU (or some nonlinearity), stacking linear layers would collapse back into a single equivalent linear transform, so ReLU's role is to let this up-projection (and the following down-projection) learn genuinely nonlinear, more expressive feature combinations rather than to perform the projection itself.
How are we projecting a lower dimension feature space into higher dimension just by using RELU?	It is mainly an architecture + compute + data trade-off, chosen before training.
for any LLM how do someone decide how many dimension are good enough?	The "right" hidden dimension is not determined by the vocabulary size or by a fixed rule. It is an empirical design choice driven by model capacity, number of layers/heads, training-data size, and available compute.
	Scaled dot-product attention is the core calculation inside the Attention block of a Transformer. It happens after we create Q, K, V and before the outputs of the attention heads are combined. So "scaled dot-product attention" is not a separate layer sitting somewhere else. It's the mathematical operation performed inside each attention head.
Where is Scaled dot product attention applied in transformer architecture?	Multi-Head Attention = multiple scaled dot-product attention operations running with different learned Q/K/V projections.
Softmax is applied just once in the architecture in the decoder side is that correct?	That's not correct. Softmax is applied multiple times: in every self-attention and cross-attention head in both the encoder and decoder (to normalize attention scores), plus once more at the very end of the decoder to convert output logits into a probability distribution over the vocabulary.

How does sentence completion is worked in parallel by transformer model as every next word will depend on previous one	During training, the whole target sequence is fed in at once with causal masking so every position's next-word prediction can be computed in parallel (each position only attends to earlier ones, mimicking sequential dependency without needing to generate step-by-step); but during actual inference/generation, the model still has to produce tokens one at a time since each new word truly depends on the previous ones being known. Transformers still have real limitations: self-attention's compute and memory scale quadratically with sequence length, making very long contexts expensive, and they lack an inherent notion of order or recurrence (needing positional encodings as a workaround) or true persistent state, plus they have no built-in inductive bias for things like recursion or algorithmic reasoning. This has driven newer research into more efficient/linear attention variants, state-space models (e.g., Mamba), and hybrid architectures. So it's an evolving space, not a solved one. Because training already knows the entire target sequence, while generation doesn't.
we have seen each of these techniques RNN, LSTM, Transformers have been developed to solve a limitation of the previous generation. Are there any limitations of transformers with self attention or it has cracked it all?	Exactly, translation is not simply word-by-word matching. That's the key insight. Translation is therefore: Source sentence?meaning/context representation?target-language generation
Why is a Transformer highly parallel during training but still sequential during text generation?	The 10000 in the original Transformer positional encoding is basically a scaling constant used to create sinusoidal waves with many different frequencies.
In case of translation, the input words gets matched to translated language words based on probability. But how does the same meaningful sentence is reconstructed in translated language as the sentence framing is different in different languages.	Yes. For streaming or very long input, positional encoding itself is only part of the problem. The bigger issue is that the model's context and attention cost keep growing. positional encoding can be extended, but for truly long/streaming data, the more important optimization is controlling the amount of context the Transformer needs to retain and attend to.
why is it 10000 in the PE formula?	Extending positional encoding does not automatically make the model good at understanding arbitrarily long contexts.
What will happen with postional encoding when we have a streaming data or very long input? How we have to optimize the encoding?	Position encoding doesn't need to be monotonic. What matters is that the encoding preserves enough information about position—especially relative position and order—for attention to distinguish relationships between tokens.
does the relation between the actual position and the encoding always have to be monotonic?	

Dimension of Positional embedding should be same as the word_embedding?	Usually yes, in the standard Transformer architecture — if you are adding positional embeddings directly to token/word embeddings, they need to have the same dimension. If positional embeddings are being added element-by-element to word embeddings, they must have the same dimension. But positional information doesn't inherently need to have the same dimension; architectures like RoPE inject position differently.
why positional encoding needed? what drawback it is covering from previous model	self-attention has no idea about word order. Positional encoding is needed because self-attention itself is permutation-insensitive — it doesn't inherently know the order of tokens. RNNs got order naturally from sequential processing, but Transformers removed that sequential dependency for parallelism, so positional information has to be explicitly injected.
Then can a word coming later in the sequence have a higher encoding value compared to a previous word? if yes, then for what possible reasons?	Yes. Absolutely. A later word can have a higher positional-encoding value than an earlier word — and in many positional-encoding schemes, this happens naturally. The key is that "position" and "encoding value" are not the same thing. Later position ? necessarily higher encoding value.
will inferencing technique be also covered?	we will cover the autoregressive generation but a bigger view on how to change the inference like varying temp, beam search, top_k, top_p etc will be covered in the subsequent lectures
Let me rephrase my question- simply put, what other information the model is learning in the encoding apart from the tokens own position?	Positional encoding is primarily giving the model information about position and relationships between positions. It isn't secretly encoding another semantic meaning like "this is a noun" or "this is important." Apart from the token's own position, positional encoding primarily provides information from which the model can learn order, relative distance, and directional relationships between tokens. It isn't intended to encode additional semantic information.
Sir how come same word will have different embedding unless and until the word has got some extra tense...is it only true in positional encoding	The same token starts with the same embedding. Positional encoding can make its initial representation position-dependent, and self-attention then makes its representation context-dependent. Tense doesn't have to be the reason for the embedding to differ.

Once we add positional encoding is getting added vector's value is getting changed. does not it affect attention score	Yes, positional encoding changes the token representation, so it also affects the attention score. Since Q and K are computed from the combined vector token embedding + positional encoding, the resulting QK ^T score contains both semantic information and positional information. This is intentional, because it allows the Transformer to consider not just which words are related, but also their order and position in the sequence.
Why are we adding Positional embedding factor in K as well and not only on query (Q)? What advantages does it have in terms of self attention?	Adding positional information to both Q and K allows the attention score itself to depend on the positional relationship between the two tokens. If only Q had positional information, the query would know its own position but would have much less direct information about where the token it is attending to is located.
In this context, what is hallucination? Where does that get injected?	Hallucination is fluent but factually wrong output, since the model predicts plausible next tokens, not verified truth. It "gets injected" at sampling time, when softmax assigns real probability to a wrong-but-plausible token and it gets picked.
What are the cost concerns of these schemes ?	The main cost difference is in memory, computation, and ability to handle longer sequences. Standard learned/sinusoidal positional embeddings are very cheap because they are simply added to the token embeddings, but learned embeddings may require a fixed maximum sequence length.
ROPE and ALiBi concept diff to understand pls explaining more or give resources to understand in depth	otod will share some additional resource in the post read
which scheme should be used for RAG	For a Transformer used in RAG, RoPE is usually a strong choice, especially when the retrieved context can be long. It works well with modern decoder-based LLMs and handles relative positions naturally. ALiBi can also be useful for very long contexts because it adds almost no extra parameter cost and can extrapolate well to longer sequences. So, practically: RoPE is the common default for modern LLM-based RAG; ALiBi is attractive when long-context efficiency and extrapolation are especially important.
Why do we need to inject positions?	We inject position information because self-attention by itself does not know the order of tokens.
Transformer is not autoregressive since there is parallel processing is it correct?	partially, encoder can be parallelized but what about the decoder. so there are three types of model -> encoder only, decoder only, and encoder and dec (will be covered in few minutes). So in dec only model during inference its is not possible to parallelize..
What is <EOS>?	End of sentence

How does LLMs handle incorrect inputs like typos, mis-spelling, incorrect grammar etc	LLMs handle them surprisingly well because they don't require an exact dictionary match for every word. Several mechanisms help. 1. Tokenization handles unknown/misspelled words 2. Context helps recover the intended meaning 3. Grammar mistakes don't necessarily prevent understanding 4. Probability helps choose the likely interpretation
What is the bottleneck that india faces in creating its own AI models just like china created Qwen.?	Good question. one reason is the resources and another can be lack of multi-lingual data
Limitation of transformer and attention?	Self-attention's compute/memory cost grows quadratically with sequence length, and transformers have no inherent sense of order, state, or grounded truth, relying on positional encodings and pattern prediction rather than true reasoning or verification.
why FFN in decoer is 4 times size of dimension?	4x is an empirical architecture choice that gives the FFN substantially more capacity while projecting back to the original dimension required by the residual connection.
does model distillation also require high resources? can you give an idea like what india lacks hardware or the cost for model development?	Model distillation does require compute, but usually much less than training the original large model from scratch.
Are the Gemini and Claude family of models also decoder only like GPT	Gemini is decoder only, but in case of claude, anthropic does not officially release the architecture nuance, but most probably is a decoder moel
if GPT is only decoder based model how RAG things also worked in GPT	RAG does not require an encoder-decoder Transformer. The retrieval part happens outside GPT: a separate embedding/retrieval system first finds relevant documents, and then those retrieved passages are inserted into GPT's prompt along with the user's question. GPT, even though it is decoder-only, simply reads that combined context and generates the answer autoregressively
what is chinchilla kaplan scaling law	The Kaplan scaling law (2020) suggested model performance scales predictably with compute mainly by growing model size faster than data, while the Chinchilla law (2022) corrected this, showing optimal performance requires scaling model size and training data roughly equally that means most large models of that era were actually undertrained relative to their size.
since decoder based models do not have context understanding they just predict next token based on training, why not industry scales BERT like enoder based model or encoder decoder model for AGI like requirements where NLU is essential	Decoder-based models are trained using next-token prediction, but predicting the next token requires modeling the context. Causal self-attention allows the model to use all preceding context, so decoder-only models can develop strong NLU capabilities even though their training objective is generative.

Which architecture is used for Agentic AI use cases out of the three?	Agentic AI use cases almost exclusively use decoder-only Transformer architectures (like GPT-style models), since their autoregressive, causal generation naturally supports the iterative reasoning, tool-calling, and multi-turn decision-making that agents require. RNNs and LSTMs lack the parallel training efficiency and long-range context handling needed at scale.
Please provide one example of how efficiency is being improved in the Decoder models in real time or industry	practical industry example is KV caching during LLM generation KV caching is a major real-world efficiency optimization for decoder-only LLMs: instead of recomputing attention's Key/Value representations for the entire previous context at every generated token, the system caches them and only computes the new token's contribution.
But can we develop an RAG based system without any model, as i am new to this ecosystem just curious to know	Not really. RAG (Retrieval-Augmented Generation) inherently needs a generative model to produce the final answer, though you could build just the "retrieval" half (a search/lookup system over documents) without any generative model, but then it wouldn't be RAG, just plain information retrieval.
This sounds like memoization in recursion Aakash.	KV caching is conceptually similar to memoization in recursion.
Do Encoder-Decoder models make more sense in specific usecases with bounded context like questions on Constitution or law interpretation that are present. Or will it be just preparing different set of decoder models for different use-cases?	the type is dependent on the task not the domain.